PromoLens

Message the customers an offer actually changes.
Stop paying to reach people who would have bought anyway.

ExaThon India 2026 | Track 02: Customers, Channels & Demand

Uplift modelling on real randomised campaigns, scored and measured inside Exasol

The problem: campaigns reach the wrong people

Who has it, and why it matters

Who has it

CRM and marketing teams at retailers and consumer brands.

They send SMS, WhatsApp and email campaigns to millions of customers and pick the audience by who is likely to buy.

What goes wrong

Likely buyers are often people who would have bought without the message.

In the X5 grocery test, 89% of messages sent to the likeliest buyers go to people the model expects to buy anyway.

Why it matters

Every unneeded message costs send volume, customer attention and opt-outs.

Reach should go to customers the message changes, and a team needs proof that it does.

The idea: four kinds of customer

Ranking by who is moved, not who is likely to buy

Offer tips them

Buy with the message, not without it. Worth contacting.

Would act anyway

High chance either way. The message adds little.

Won't act either way

Low chance either way. The message adds little.

Offer puts them off

Less likely to buy after the message.

Uplift = chance of buying with the message minus chance without it.

A person's own effect is never observed. Randomised experiments let us estimate it from the difference between messaged and not-messaged customers who look alike.

Groups and "would buy anyway" are model estimates, labelled as such everywhere in the product.

What we built, end to end

Exasol does the data work. Python only trains a small model and serves the API.

1 Load

Three public randomised campaign datasets into typed Exasol tables. 45.8M purchase lines for X5.

2 Features

Built in SQL: recency, frequency, spend, baskets, stores, categories, points. 25 features for X5.

3 Split + train

70/30 split by SHA1 hash of client id, in SQL. Two logistic models on the 70%: messaged and not.

4 Score in Exasol

Coefficients stored in a table. A SQL view computes p0, p1 and uplift for every customer.

5 Evaluate

Held-out 30%: three strategies, deciles, Qini, bootstrap 95% bands.

6 Serve

FastAPI + React. Every number carries its SQL and query_id.

`Local Exasol (Docker) > pyexasol > FastAPI > React with custom SVG charts`

Data: three public randomised experiments

Customers were assigned to receive a message or not at random, so the difference is causal

Dataset	Campaign	Outcome	Customers	Held out (30%)	Rate: messaged vs not
X5 RetailHero	SMS, grocery chain	Purchase afterwards	200,039 labelled + 45.8M purchase lines	60,214	63.4% vs 60.1%
Lenta	SMS, grocery chain	Store visit	687,029	206,133	11.0% vs 10.1%
Hillstrom	Email, online retailer	Purchase in 2 weeks	64,000	19,446	1.06% vs 0.41%

Public research data from Russia and the US. The product reads each dataset from a config, so adding a new campaign needs no new API or UI code.

Counts and rates only: no currency or revenue figures anywhere.

Method: simple enough to run as SQL

Explainable, reproducible, and fast on the data platform

Two-model uplift

L2 logistic regression on messaged customers (p1) and on the rest (p0). Uplift = p1 - p0. C chosen by 3-fold CV inside the training split.

Scoring inside Exasol

Coefficients, means and stds sit in a table. A view computes z-scores and probabilities with EXP for every customer.

Leakage control

Deterministic 70/30 hash split. Fit and tune on 70% only; the 30% holdout is untouched until evaluation. Purchase history ends before the campaign.

Honest uncertainty

200 bootstrap resamples of the scored holdout (fixed seed) give 95% bands on every curve and headline.

Parity test: SQL scores equal scikit-learn predict_proba. Max difference on 60,214 X5 holdout customers: 2.0e-14 (tolerance 1e-6).

Exasol at work

Measured on the local Docker deployment, from the pipeline logs

45.8M

X5 purchase lines streamed into Exasol in 256 MB chunks, never loaded into pandas

~70 s

to build all X5 purchase features in SQL (8.0M transaction rows, 16 buckets)

2.3 s

to score 400,162 X5 customers in SQL (10.0M feature values unpivoted in 2.2 s)

22.7M

feature values for Lenta's 687,029 customers scored with the same SQL

63 ms

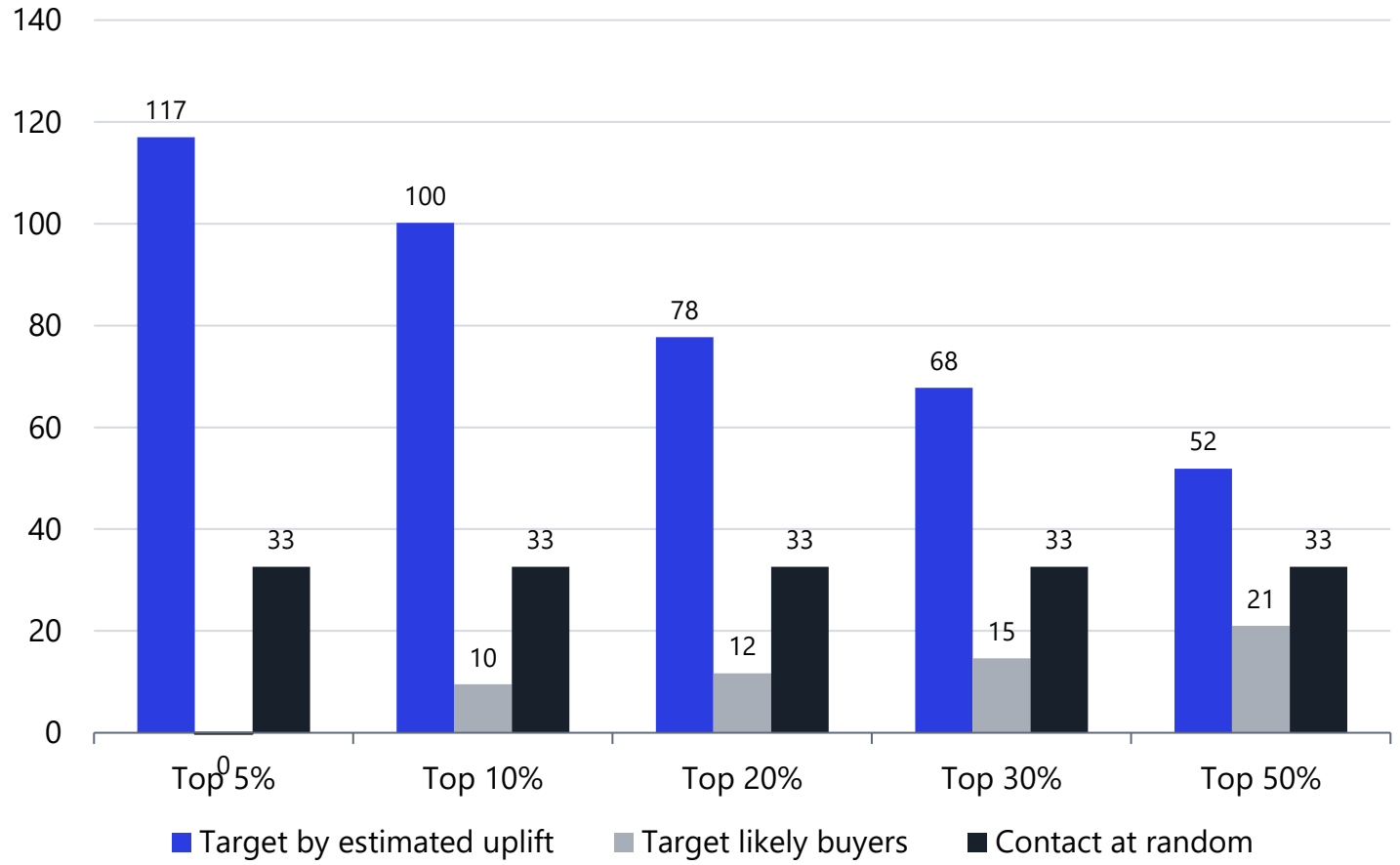
to compute the X5 decile table on the holdout

1 view

computes p0, p1 and uplift for every customer. No model server

Result: X5 RetailHero SMS

Extra purchases per 1,000 customers contacted, measured on 60,214 held-out customers



At 20% contacted

Strategy	Extra per 1,000	95% band
Uplift model	+77.7	[+62.1, +94.6]
Likely buyers	+11.7	[+0.8, +21.1]
Random	+32.6	[+25.8, +40.7]

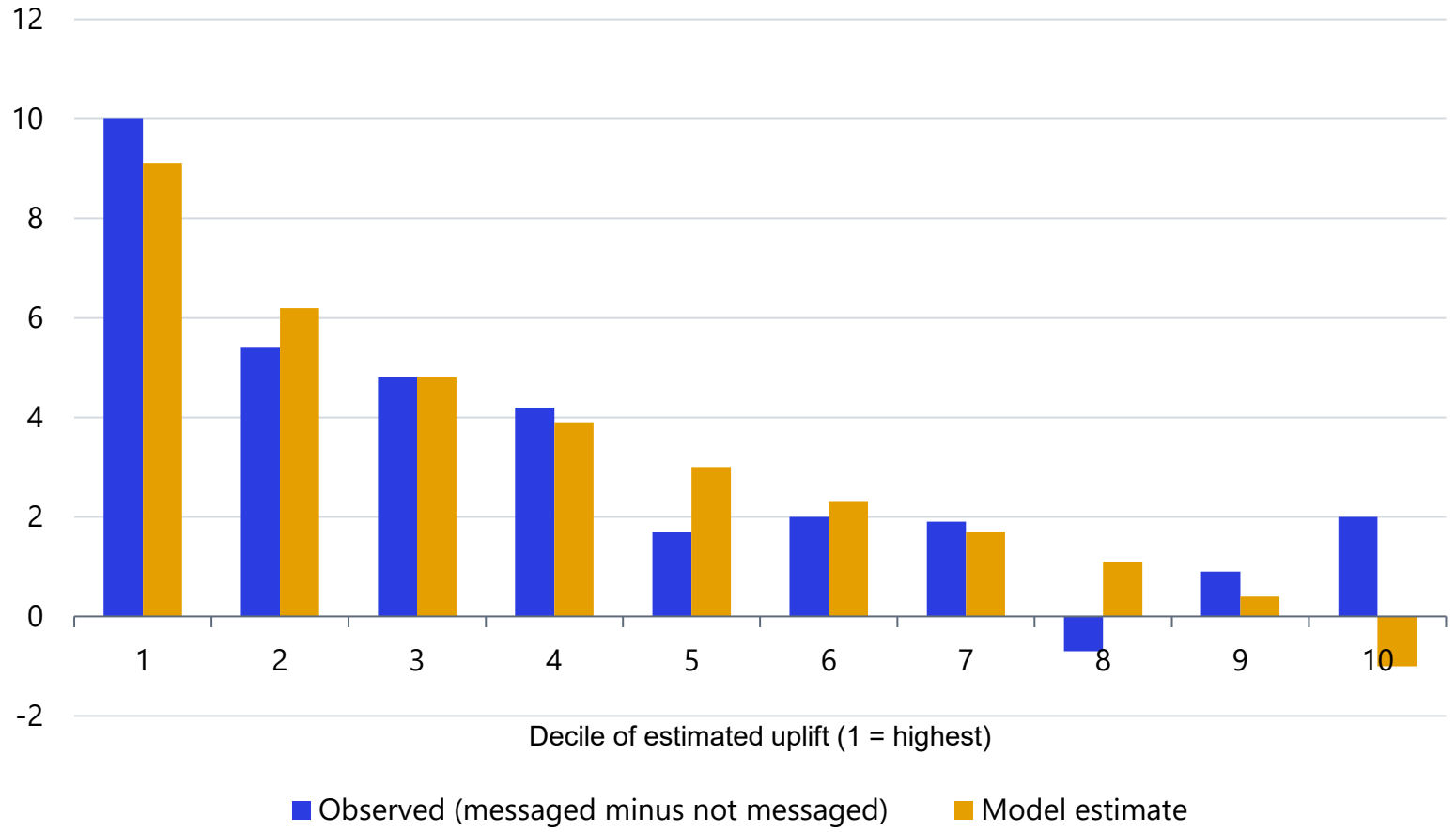
Uplift minus likely buyers: +66.1 per 1,000, band [+49.3, +85.0]. Above zero.

Uplift minus random: +45.2 per 1,000, band [+30.0, +59.2]. Above zero.

Est. share who would buy anyway: 46% (uplift) vs 89% (likely buyers).

The ranking holds on customers the model never saw

X5 holdout, ten equal groups by estimated uplift: observed effect of the SMS in percentage points



Top decile: the SMS lifted purchases by +10.0 points (band +7.6 to +12.4), close to the +9.1 the model estimated.

Observed effects fall as the estimated uplift falls: the model orders customers the way the experiment confirms.

Bars are observed from the randomised split, not from the model.

Results across the three experiments

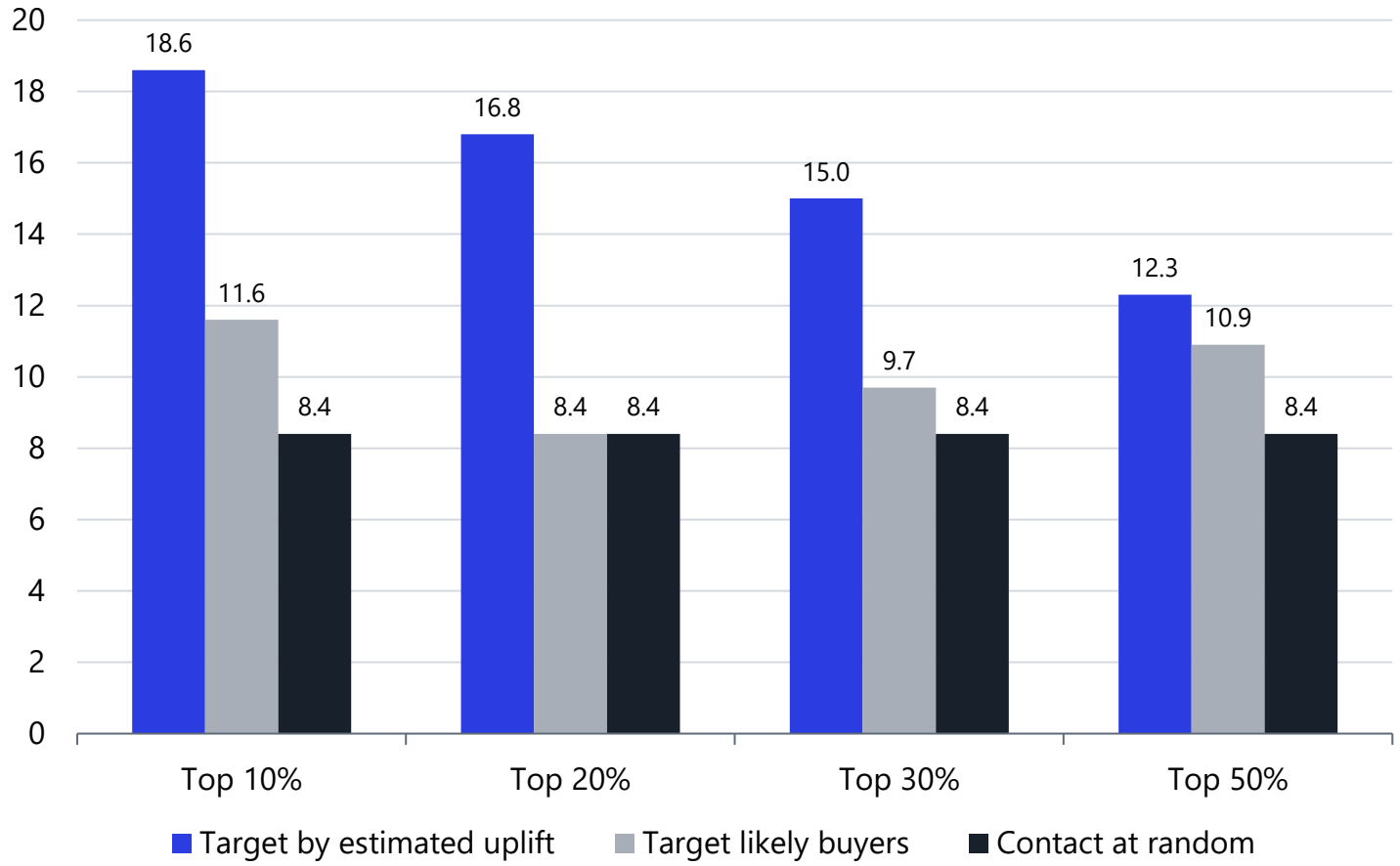
Same pipeline, same SQL, same method. Top 20% contacted, held-out customers, 95% bands

Dataset	Uplift model, extra per 1,000	Likely buyers	Random	Uplift minus likely buyers	Uplift minus random
X5 RetailHero (purchases)	+77.7 [+62.1, +94.6]	+11.7 [+0.8, +21.1]	+32.6 [+25.8, +40.7]	+66.1 [+49.3, +85.0] better	+45.2 [+30.0, +59.2] better
Lenta (store visits)	+16.8 [+9.1, +26.0]	+8.4 [-1.1, +20.8]	+8.4 [+5.6, +11.7]	+8.4 [-0.6, +17.6] inconclusive	+8.4 [+2.1, +16.2] better
Hillstrom (purchases)	+7.9 [+2.4, +13.8]	+8.1 [+2.1, +14.2]	+6.5 [+4.1, +8.8]	-0.3 [-5.7, +4.2] inconclusive	+1.4 [-3.5, +6.3] inconclusive

Bands that stay above zero are called better; bands that include zero are called inconclusive in the product. Hillstrom's effect is small (under 1.1% buy), so it shows the pipeline running end to end on a third dataset.

Result: Lenta SMS

Extra store visits per 1,000 contacted, measured on 206,133 held-out customers



At 20%: +16.8 extra store visits per 1,000 [+9.1, +26.0] vs +8.4 at random.

Uplift minus random: +8.4 per 1,000, band [+2.1, +16.2]. Above zero.

Uplift minus likely buyers: +8.4 per 1,000, band [-0.6, +17.6]. Points the same way; the band touches zero.

Share of contacted who would visit anyway (est.): 18% by uplift vs 29% by likely buyers.

Benchmark: which model do we ship?

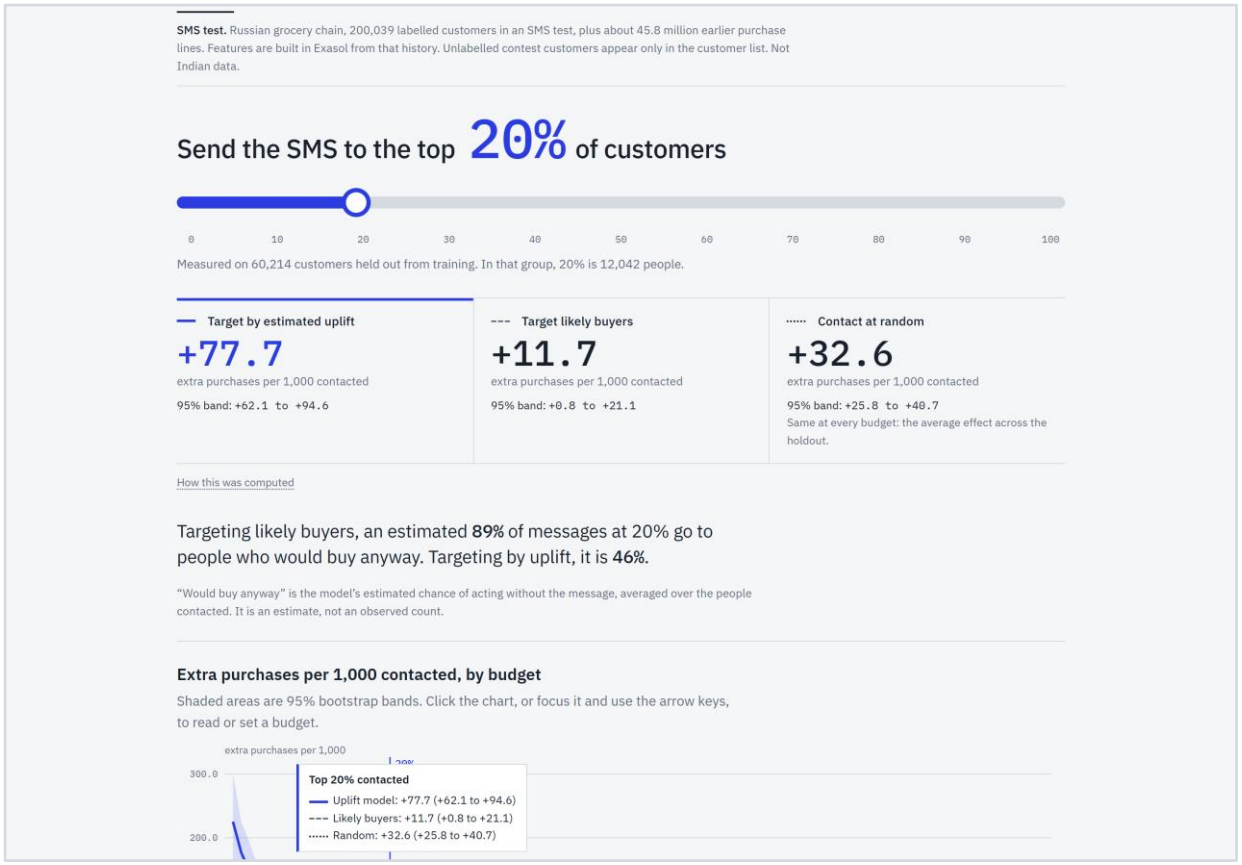
Five-fold out-of-fold Qini on the Lenta training split only. The holdout was never touched.

Model	Qini	Extra visits per 1,000 at 20%	Scores inside Exasol with EXP
Two-model logistic (shipped)	+0.0096	+10.2	Yes
Interaction logistic	+0.0096	+12.3	Yes
Binned features + logistic (5 bins)	+0.0095	+13.0	Yes
Binned features + logistic (8 bins)	+0.0062	+11.4	Yes
Boosted trees (offline reference)	+0.0118	+14.6	No
Likely buyers (rank by p1)	+0.0056	+7.8	Yes

We tested richer logistic variants and boosted trees, then kept the two-model logistic: it ties the best alternatives that Exasol can score, ranks above likely buyers on Qini in cross-validation, and scores entirely in SQL with a parity test against scikit-learn.

The product: drag a slider, see the evidence

Overview screen, X5 RetailHero



Offer to the top N% slider drives three strategy cards: extra purchases per 1,000 contacted, each with a 95% band.

One headline: the estimated share of messages that go to people who would buy anyway.

The curve is fetched once and the slider interpolates it, so it responds instantly.

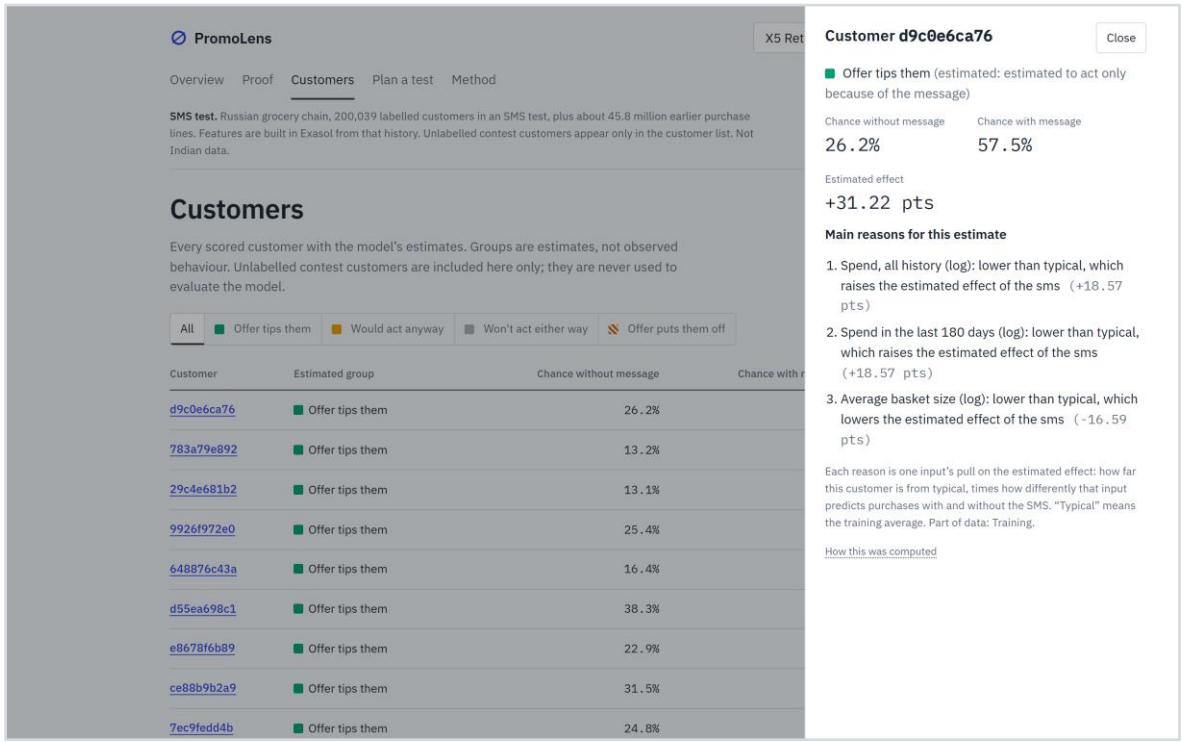
A stacked bar shows who you would contact, with every group colour paired with a text label.

Light and dark themes, responsive down to 360 px, keyboard-reachable charts.

Proof, and the customers behind it

Proof screen and customer drawer

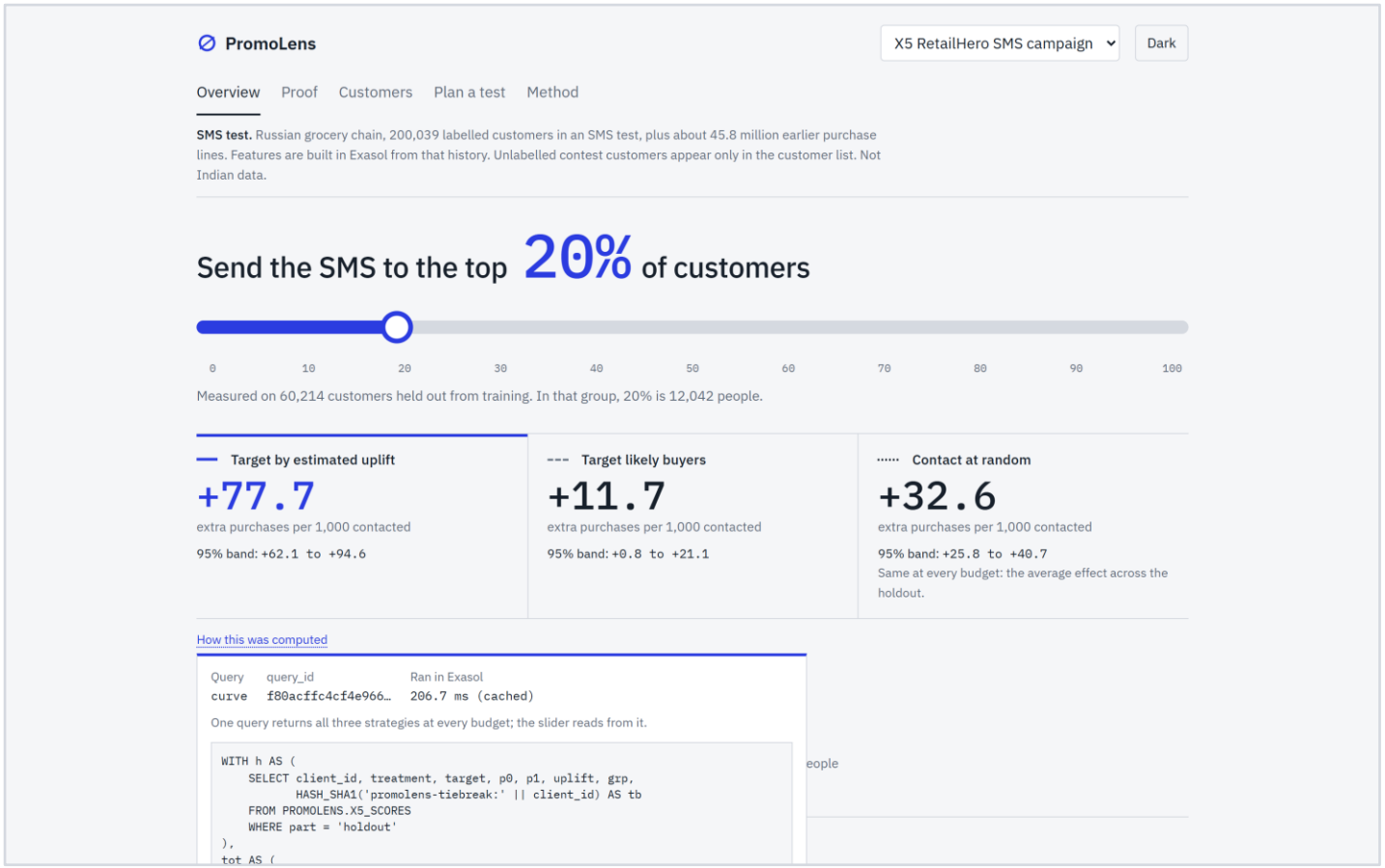




Left: head-to-head with bands, decile bars, whether the estimated groups behave as labelled. Right: three plain-language reasons per customer, from coefficient times z-score.

Every number can show its work

"How this was computed" on each metric



Every metric returns its SQL, a query_id (sha1 of the SQL text) and the time it took in Exasol.

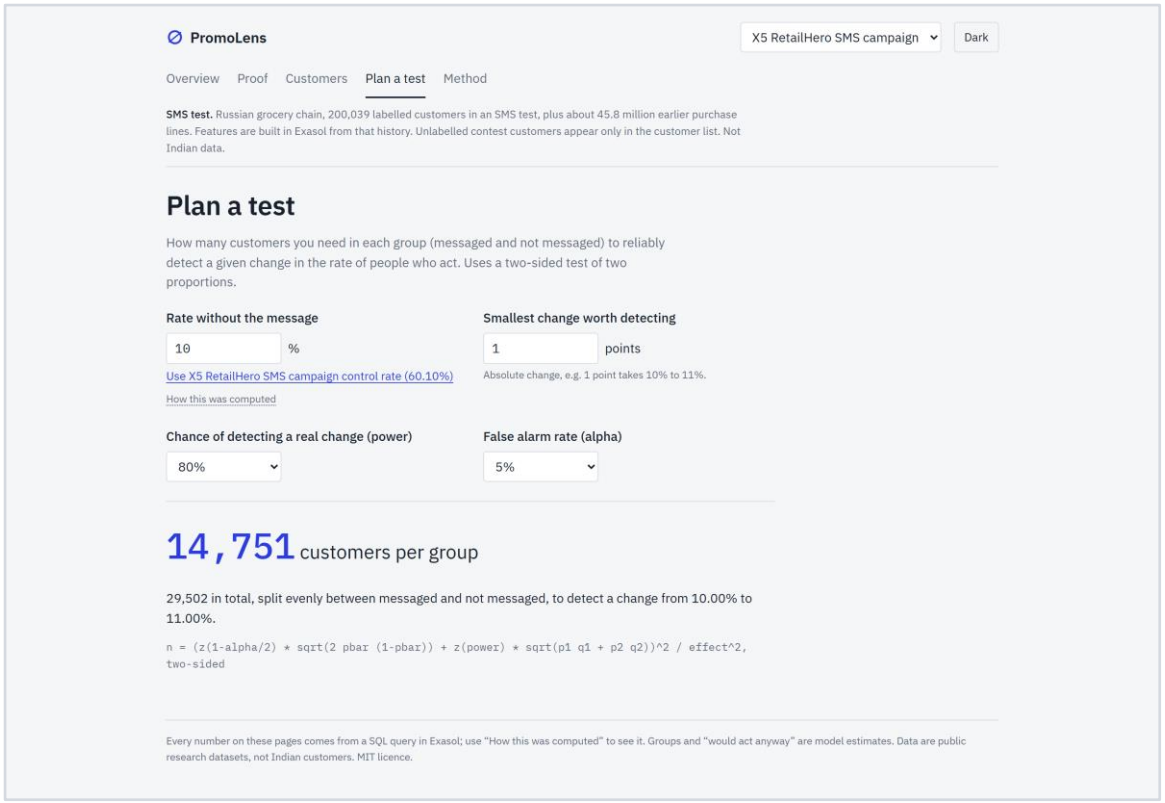
Each query is also logged to PROMOLENS.QUERY_LOG with the session id.

No hardcoded or mock numbers in the UI.

Estimates are labelled as estimates; a band that includes zero is shown as inconclusive.

From estimates to a confirmation test

Plan a test: customers needed per arm



Pick the baseline rate, the smallest effect worth detecting, power and significance level.

The calculator returns customers needed per arm for a two-sided two-proportion test.

Example: a 10% baseline, a 1 point lift, 80% power and 5% alpha need 14,751 customers per arm.

Use it to size a fresh randomised test on the audience the model selects.

Run it anywhere Docker runs

Reproducible from the public repository

Setup

1 Start Exasol (exasol/docker-db), pin its TLS fingerprint in .env

2 python -m pipeline.doctor

3 python -m pipeline.all --dataset hillstrom | lenta | x5

4 Run the API and UI in dev, or build one image

Tests: 30 pass, ruff clean, SQL-vs-sklearn parity included

Deploy the app

docker build -t promolens-app .

docker run -p 8000:8000 --env-file .env promolens-app

One container serves the API and the built UI on port 8000.

TLS verification stays on: the certificate fingerprint is pinned in the DSN.

MIT licence. Public datasets only. No paid APIs or keys.

Message the people an offer changes.

On X5 held-out customers, targeting by uplift gave +77.7 extra purchases per 1,000 contacted at 20%, versus +11.7 for likely buyers and +32.6 at random.

Trained on randomised experiments. Scored, evaluated and explained inside Exasol.

Three datasets through one adapter. Every number traceable to its SQL.

PromoLens | public repository on GitHub | MIT licence